

SecureCam: Zero-Trust Surveillance Access System using PAKE Authentication

Network Programming and Security Lab (CS362IA)



Presentation Overview



01

Introduction & Problem

Security gaps in traditional surveillance and our zero-trust approach



02

System Architecture

React frontend, FastAPI backend, and the zero-trust security chain



03

Security Design

PAKE authentication, capability tokens, RBAC, and replay protection



04

Implementation

Database design, frontend/backend modules, and security monitoring



05

Results & Conclusion

Testing outcomes, acknowledged limitations, and future enhancements



01

Introduction & Problem

Understanding the security gaps in traditional surveillance systems and defining our zero-trust approach





Problem Statement

Problems in Traditional Systems

- **Password exposure risk** — credentials may be intercepted or stored insecurely if authentication is weak
- **Session token misuse** — a login JWT may be treated as permission to view camera feeds
- **Over-authorization** — one successful login can expose multiple sensitive cameras
- **Permanent access** — granted permissions may remain active without expiry
- **No approval workflow** — users may access sensitive feeds without admin validation

Our Solution Requirements

- ▶ Secure authentication without plaintext password transmission (**PAKE**)
- ▶ Identity tokens that do **not** imply camera access (**JWT identity only**)
- ▶ **Least-privilege access** with camera-specific, time-bound grants
- ▶ **Admin approval** before sensitive access
- ▶ **Nonce-based replay protection** for capability validation

Core insight: Traditional surveillance portals treat **login as authorization** — once authenticated, users gain broad or permanent access. Our system separates **authentication**, **authorization**, and **accountability** through a zero-trust pipeline.



Primary Objectives

1. **Secure Authentication**

Implement PAKE/SPAKE2 login so the password is never directly transmitted.

2. **Zero-Trust Camera Access**

Ensure login alone does not unlock cameras; access requires assignment, capability token, and nonce validation.

3. **Replay-Resistant Authorization**

Prevent reused authorization requests using fresh nonces and log replay attempts.

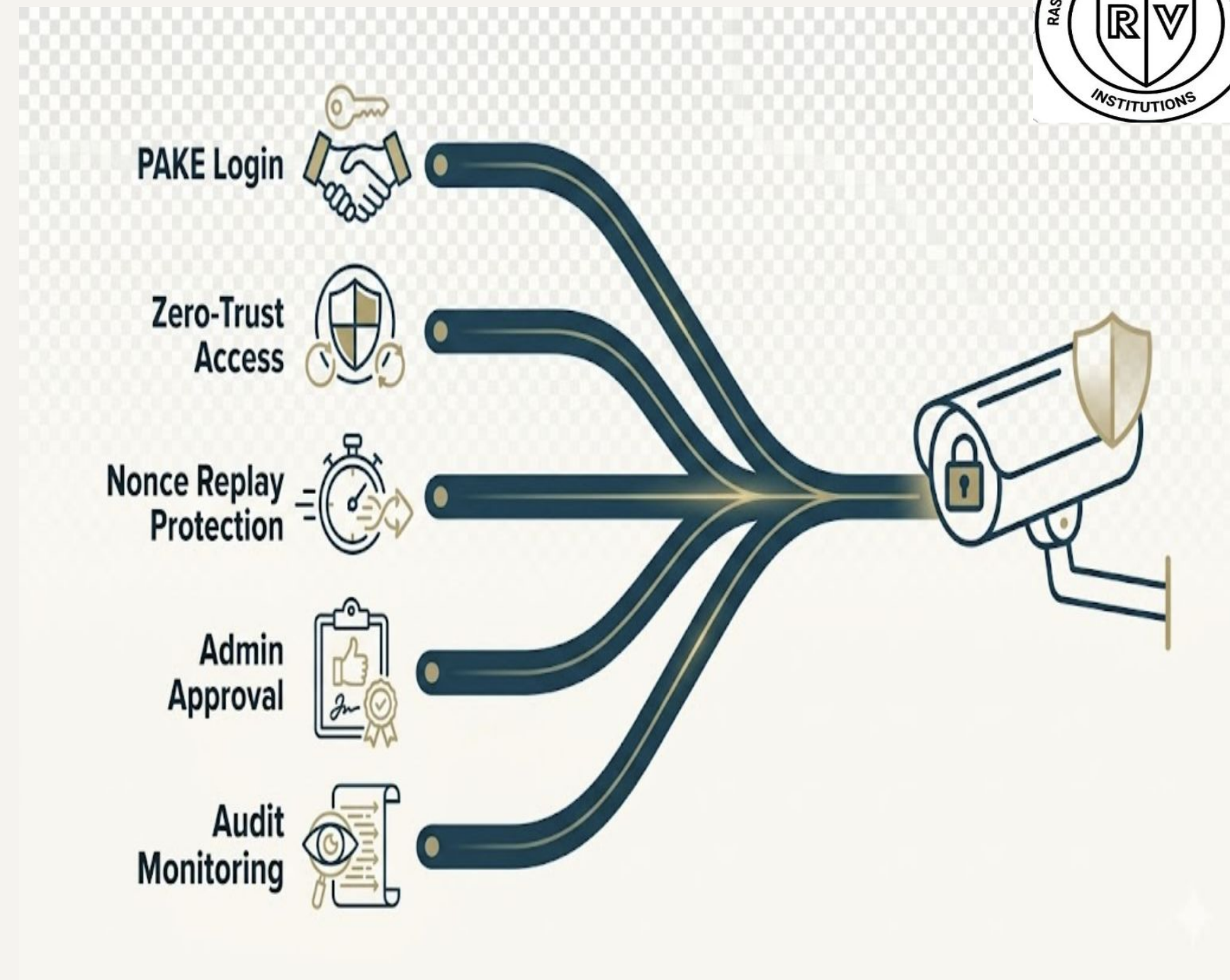
Secondary Objectives

4. **Admin-Controlled Access Management**

Support request approval, time-bound assignments, revocation, and role-specific dashboards.

5. **Auditability and Security Monitoring**

Maintain audit logs, security events, and dashboard metrics for traceability.



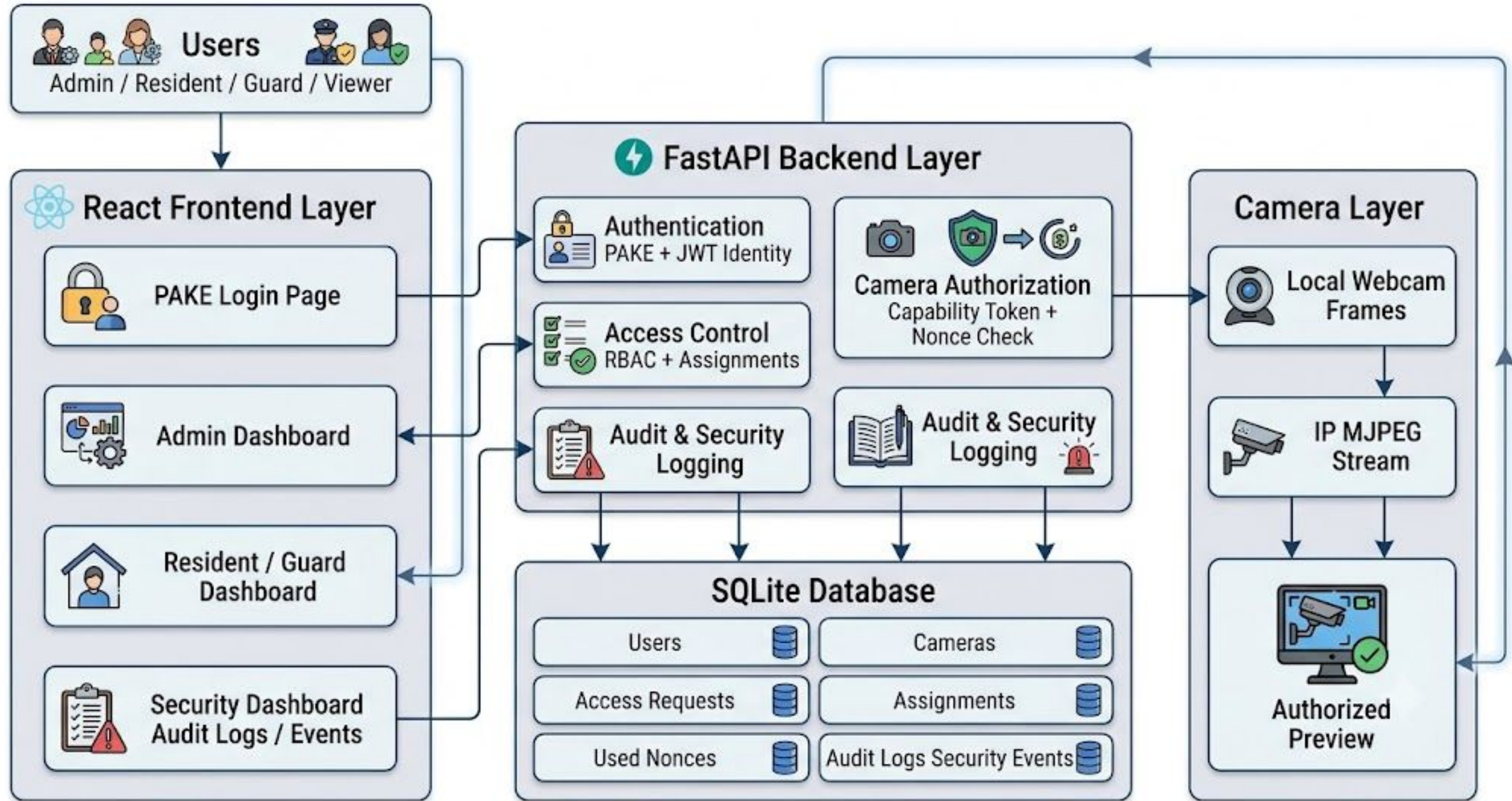
02

System Architecture

React frontend, FastAPI backend, SQLite database, and the zero-trust security chain



System Architecture Overview





System Architecture Overview

Frontend (React + Vite + TS)	Backend (FastAPI + Python)	Database (SQLite nps.db)
■ Landing page + PAKE login UI ■ Role dashboards (admin, resident, guard) ■ Security Dashboard + Audit Logs ■ lib/pake.ts (SPAKE2 client) ■ lib/api.ts (HTTP client)	■ main.py — All HTTP routes ■ auth.py — JWT + Password hash ■ access.py — RBAC + Capabilities ■ audit.py — Events + Logs ■ pake_bridge.py — SPAKE2 bridge	■ users (PAKE salt/verifier) ■ cameras + access_requests ■ assignments (active/expired/revoked) ■ used_nonces (replay protection) ■ audit_logs + security_events
Camera Layer	Technology Stack	
admin_local (webcam) viewer_local (owner webcam) ip_mjpeg (network proxy) unconfigured	React + Vite (Frontend) FastAPI + SQLAlchemy (Backend) SQLite SPAKE2 JWT Argon2id	
Design principle: The frontend handles UI and PAKE computation; the backend enforces all security policy (RBAC, capabilities, nonce validation); SQLite provides durable state for audit trails.		



End-to-End Zero-Trust Workflow



Step	Action	Description
1	PAKE Login	User proves password knowledge via SPAKE2 — password never sent over network
2	JWT Issued	Server issues identity token (sub + role only) — does NOT grant camera access
3	Request Access	Resident/guard submits access request with reason for specific camera
4	Admin Approval	Admin reviews and approves/rejects; approval creates time-limited assignment
5	Capability Token	Camera-scoped permission token issued only when active assignment exists
6	Nonce Validation	Fresh one-time nonce consumed; replay of same nonce returns 409 Conflict
7	Camera Access	Frame polling or MJPEG stream begins with validated capability token

03

Security Design

PAKE authentication, capability tokens, RBAC, and nonce-based replay protection



PAKE Authentication

What is PAKE?

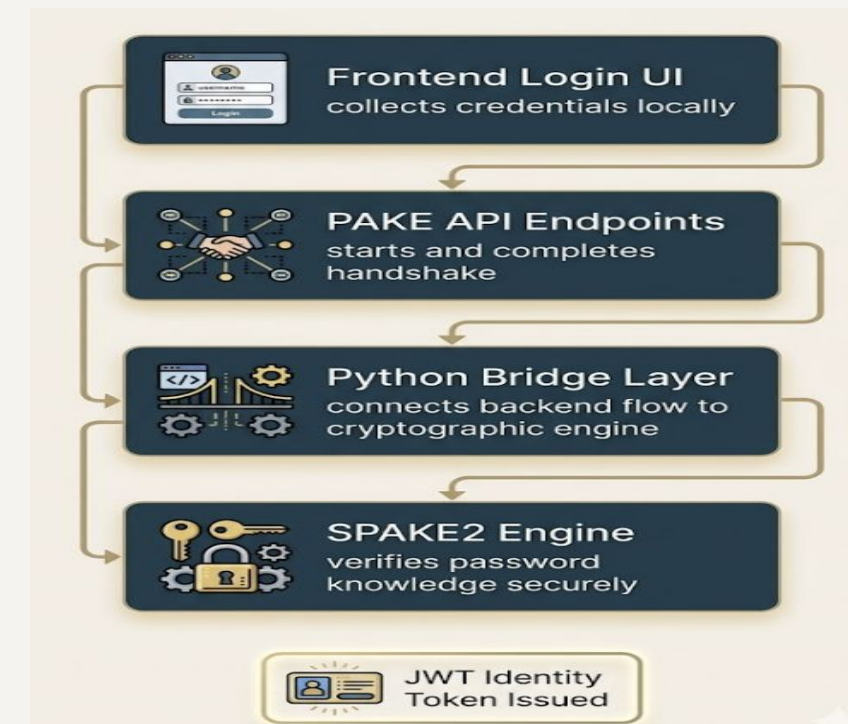
Password Authenticated Key Exchange allows two parties to prove password knowledge **without transmitting the password** over the network.

In normal login, the client sends the password to the server. If the channel is compromised, the password is exposed. PAKE eliminates this risk.

How It Works



Implementation Chain





JWT Identity vs Capability Tokens

"Who is this user?" → JWT identity token "Can this user view this camera?" → Capability token

Feature	JWT Identity Token	Camera Capability Token
Purpose	Proves user identity	Grants camera-specific access
Scope	User/session level	User + camera + assignment
Contains	sub, role, iat, exp	sub, camera_id, assignment_id, permissions, jti, exp
Used for	API authentication (Bearer header)	Camera authorization (frame/stream)
Camera-specific	No	Yes — scoped to one camera
Expiry	Session expiry (-configurable)	Tied to assignment (max 15 min)
Zero-trust role	Identity only	Least-privilege (VIEW)
If stolen	Exposes general APIs	Exposes only one camera for shorter window

Security principle: A stolen JWT lets an attacker call general APIs as that user — but it should **not** let them view assigned admin cameras without capability + nonce + active assignment checks.

RBAC and Access Control



 Admin	 Resident	 Security Guard	 Viewer
• Full system control • Manage cameras • Approve/reject requests • Revoke assignments • View audit logs • Security dashboard	• Request camera access • View assigned cameras • View owned cameras • Time-bound viewing • Capability token required • Nonce validation	• View assigned cameras • Limited duration only • No owned cameras • Requires admin approval • Capability + nonce • No log access	• View-only access • Specifically assigned • Restricted permissions • No request rights • Capability required • Monitored viewing

Assignment Lifecycle





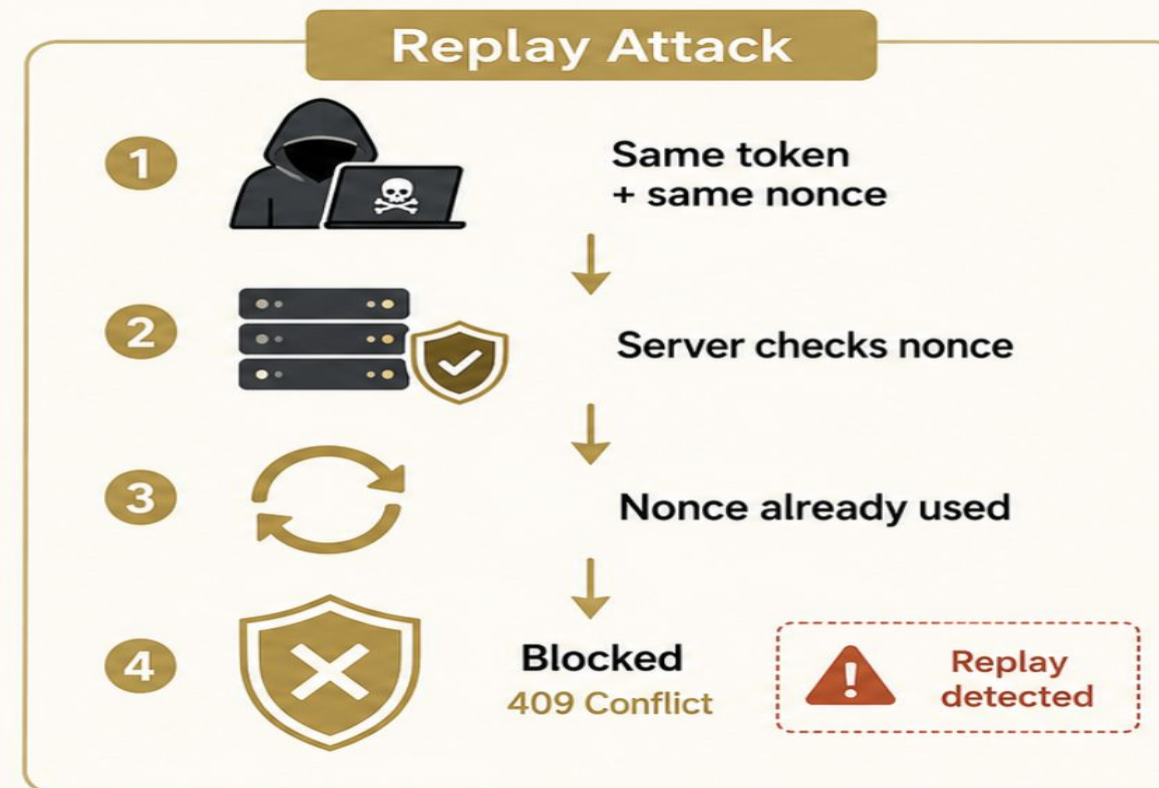
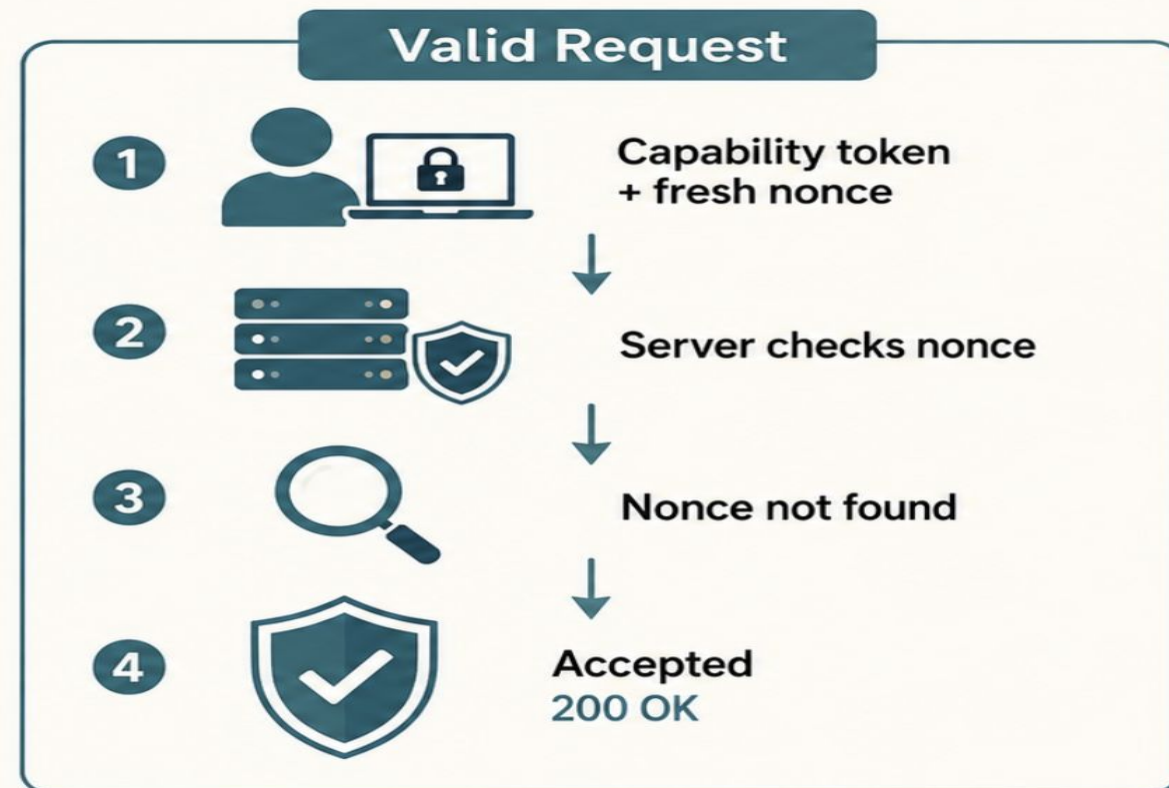
Nonce Replay Protection

What is a Replay Attack?

A replay attack occurs when an attacker captures a valid authorization request and tries to send it again to gain access. In SecureCam, this could mean resending a previously valid **capability validation request** containing a token and nonce.

Our Defense Mechanism

1. Client generates a **fresh UUID nonce** for each validation
2. Server checks used_nonces table for (user_id, nonce)
3. **First use** → nonce stored → validation succeeds
4. **Reuse** → **409 Conflict** → **REPLAY_ATTACK_DETECTED** logged



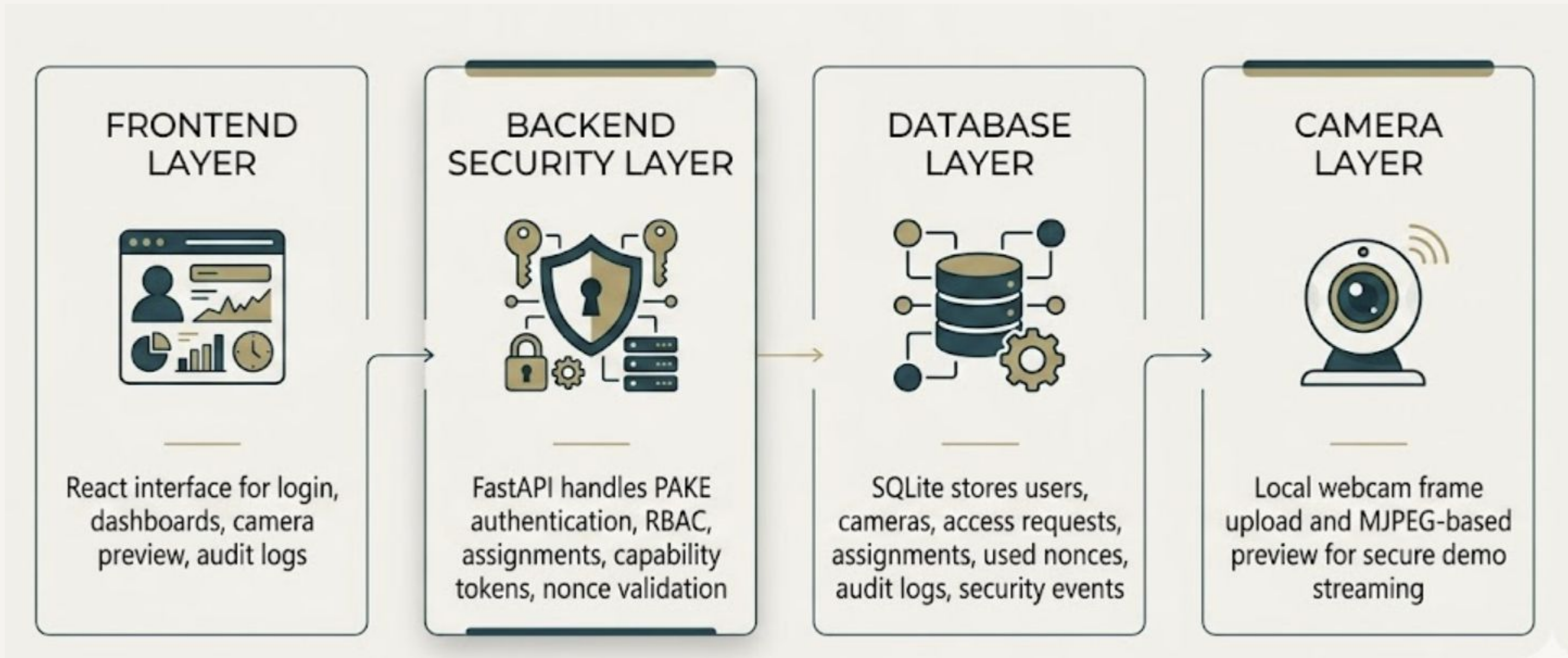
04

Implementation

Building SecureCam as a modular full-stack system where the backend enforces authentication, authorization, replay protection, and audit monitoring.



Implementation Flow



Audit Logs & Security Dashboard



Dual Logging System

AuditLog

Actor ID + target ID + description. Compliance trail for admin audit page.
Who did what and when.

SecurityEvent

Severity + category + usernames + JSON details. Security dashboard alerts and monitoring.



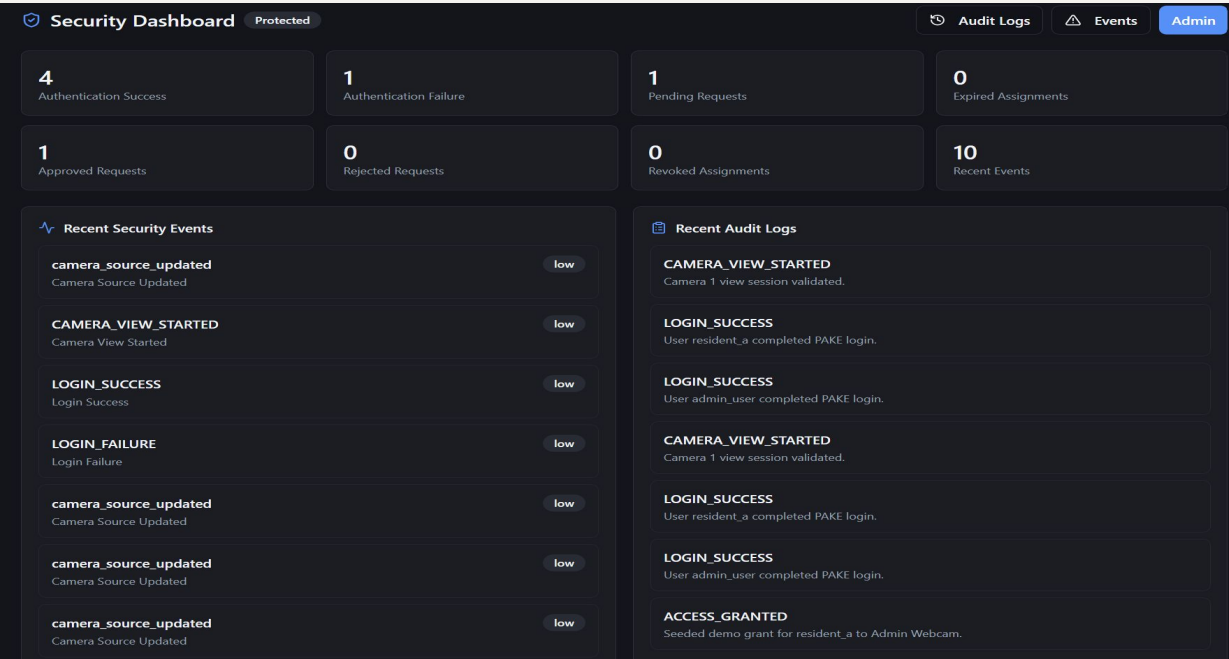
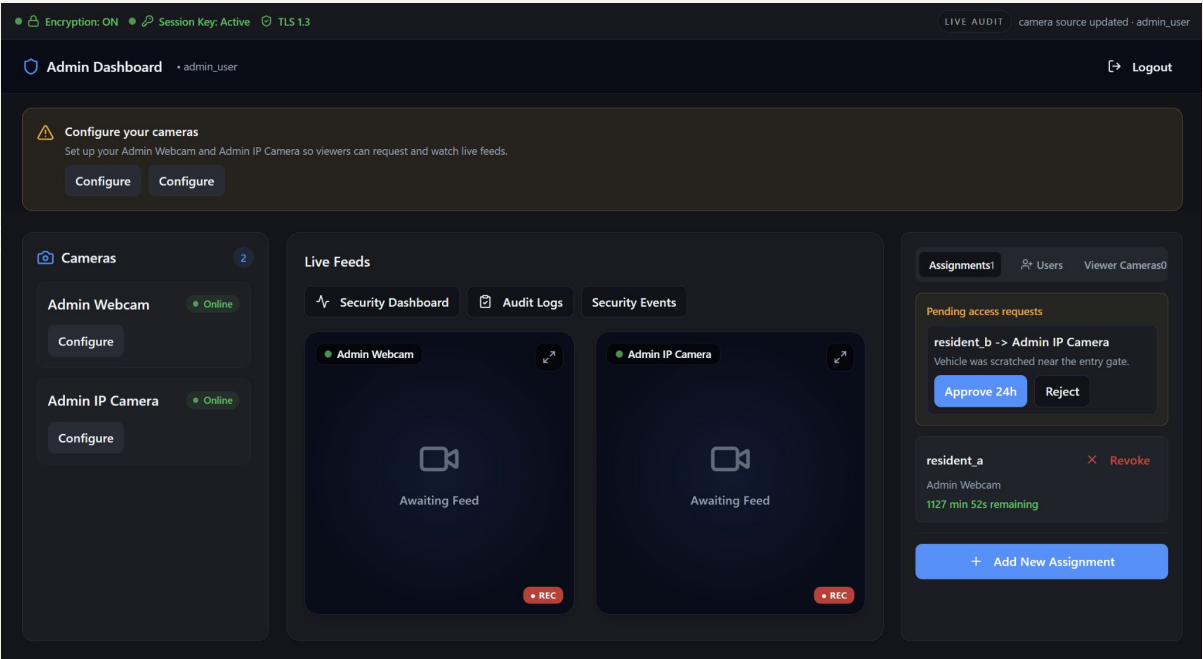
05

Results & Conclusion

Testing outcomes, acknowledged limitations, and future enhancements



Results & Verification



Manual Verification

- ✓ Role Based Login via PAKE
- ✓ Resident sees assigned camera only
- ✓ Capability issue + validate verified
- ✓ Replay same nonce → **409 Conflict**

- ✓ REPLAY_ATTACK_DETECTED in Security Events
- ✓ Security dashboard shows metrics
- ✓ Soft revoke – assignment row retained
- ✓ SQLite schema verified (all tables)



Limitations & Future Scope

Limitations

- JPEG frame polling (~500ms) — **not WebRTC**
- In-memory CAMERA_FRAMES — lost on restart
- AI analytics — **not implemented** as live page
- SQLite for local demo — not production-grade
- No rate limiting implemented
- Capability token in query string (img limitation)

Future Scope

- ▶ **WebRTC E2E** encrypted streaming
- ▶ **Real ML detection** (YOLO, etc.)
- ▶ **PostgreSQL** deployment
- ▶ **Rate limiting** per-IP and per-user
- ▶ **Mobile app** with PAKE login
- ▶ **Device binding** for tokens

We applied NPS principles — **secure authentication**, **least privilege**, **replay protection**, and **accountability** — to a real surveillance workflow, transforming access from a coarse login model into a **layered, verifiable, time-bounded authorization pipeline**.

The background is a light beige color. In the center, there is a faint, stylized shield shape. Surrounding the shield are numerous thin, light brown lines that resemble circuit traces or a stylized floral pattern, radiating outwards from the center. The lines are more dense around the shield and become sparser towards the edges.

Thank You

